

AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

i-Credit Hours Engineering Programs

Computer and Artificial Intelligence Engineering

Course Code: **CSE488**

Deep Learning

Assignment 1

MicroGrad (Scalar Autograd Engine)

Documentation Report

Student Name: Abdelrahman Mohamed Bahig

Student ID: 23P0390

Course: CSE488 – Deep Learning

Date: November 3, 2026

Contents

1 Overview **2**

2 Mathematical Derivations **2**
2.1 Reverse-mode automatic differentiation 2
2.2 Local derivatives 2
2.3 Worked example 3

3 Topological Sort and Backpropagation **3**
3.1 Algorithm 3
3.2 TA traps and how the design avoids them 4

4 Neural Network API **4**

5 Design Justifications **4**

6 Test Results **5**

7 Experiment: Two Moons **5**

8 Conceptual Questions **6**

9 Reproducibility **6**

1 Overview

This report documents a from-scratch scalar automatic differentiation engine (`src/engine.py`), a small neural network API built on it (`src/nn.py`), the course test suites (`tests/test_engine.py`, `tests/test_nn.py`), and a demo notebook (`demo.ipynb`). No PyTorch or other autograd library is used; only the Python standard library inside the engine, and NumPy/Matplotlib in the notebook for data and plots.

2 Mathematical Derivations

2.1 Reverse-mode automatic differentiation

Every arithmetic operation creates a node in a directed acyclic graph (DAG). Let L be the final scalar output and v any intermediate node. Reverse-mode accumulates $\bar{v} := \partial L/\partial v$. For a node $y = f(x_1, \ldots, x_k)$ the chain rule gives

$$\frac{\partial L}{\partial x_i} \mathrel{+}= \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x_i}. \tag{1}$$

The update uses `+=` (accumulation) because a node may feed several consumers, and by the multivariate chain rule the total gradient is the *sum* of the contributions from all consumers:

$$\frac{\partial L}{\partial x} = \sum_{y\,:\,x \in \mathrm{parents}(y)} \frac{\partial L}{\partial y}\frac{\partial y}{\partial x}. \tag{2}$$

2.2 Local derivatives

Addition. $y = a + b$: $\partial y/\partial a = \partial y/\partial b = 1$, hence $\bar{a} \mathrel{+}= \bar{y}$, $\bar{b} \mathrel{+}= \bar{y}$.

Multiplication. $y = ab$: $\partial y/\partial a = b$, $\partial y/\partial b = a$, hence $\bar{a} \mathrel{+}= b\,\bar{y}$, $\bar{b} \mathrel{+}= a\,\bar{y}$.

Power (constant exponent). $y = a^n$: $\partial y/\partial a = n\,a^{n-1}$, hence $\bar{a} \mathrel{+}= n\,a^{n-1}\,\bar{y}$. The exponent is restricted to a Python `int`/`float` constant, so no gradient flows to it.

Hyperbolic tangent. $y = \tanh a = \dfrac{e^a - e^{-a}}{e^a + e^{-a}}$. Then

$$\frac{dy}{da} = \frac{(e^a + e^{-a})^2 - (e^a - e^{-a})^2}{(e^a + e^{-a})^2} = 1 - \tanh^2 a,$$

hence $\bar{a} \mathrel{+}= (1 - y^2)\,\bar{y}$. The derivative is expressed via the already-computed output y, so no extra evaluation is needed.

Exponential. $y = e^a$: $dy/da = e^a = y$, hence $\bar{a} \mathrel{+}= y\,\bar{y}$.

Derived operators. Negation is $a \cdot (-1)$; subtraction is $a + (-b)$; division is $a \cdot b^{-1}$. Their gradients therefore follow automatically from the three primitives. For instance, for $y = a/b$: $\partial y/\partial a = b^{-1}$ and $\partial y/\partial b = -a\,b^{-2}$, which the engine reproduces through `mul` and `pow` with $n = -1$ (values checked in `test_basic_operations`; gradients checked numerically in the notebook).

2.3 Worked example

For $L = \tanh(x_1w_1+x_2w_2+b)$ with $x_1 = 2,\ x_2 = 0,\ w_1 = -3,\ w_2 = 1,\ b = 6.8813735870195432$: the pre-activation is $n = 0.8813735\ldots$, $L = \tanh n = 0.7071$, $\partial L/\partial n = 1 - L^2 = 0.5$. Therefore $\partial L/\partial x_1 = w_1 \cdot 0.5 = -1.5$, $\partial L/\partial w_1 = x_1 \cdot 0.5 = 1.0$, $\partial L/\partial x_2 = 0.5$, $\partial L/\partial w_2 = 0$. The engine reproduces these values (checked in the notebook).

3 Topological Sort and Backpropagation

A node's gradient is complete only after *every* consumer of that node has pushed its contribution. Equivalently, we must process nodes in an order where each node is handled before all of its inputs. A topological ordering of the DAG, reversed, provides exactly this.

3.1 Algorithm

```
def _get_topo(self):
    topo, visited = [], set()
    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                build_topo(child)
            topo.append(node)       # post-order
    build_topo(self)
    return topo

def backward(self):
    topo = self._get_topo()
    self.grad = 1.0                 # seed dL/dL = 1
    for node in reversed(topo):
        node._backward()
```

The submitted implementation uses an explicit stack instead of recursion (same post-order, same `visited` logic), so that very deep graphs do not exceed Python's recursion limit:

```
def _get_topo(self):
    topo, visited = [], {self}
    stack = [(self, iter(self._prev))]
    while stack:
        node, children = stack[-1]
        for child in children:
            if child not in visited:
                visited.add(child)
                stack.append((child, iter(child._prev)))
                break
        else:
            topo.append(node)       # post-order: after all children
            stack.pop()
    return topo
```

Correctness argument. Appending a node *after* recursing into all of its children (post-order) guarantees that every input appears in `topo` before any node that uses it. Reversing the list therefore visits every node before its inputs, so when `_backward` of a node runs, its `grad` has already received the contributions of all of its consumers.

3.2 TA traps and how the design avoids them

- **Post-order trap.** If a node were appended *before* its children were explored (pre-order), the reversed list could process an input before one of its consumers, which would push stale (incomplete) gradients. The composite test `test_chain_rule_and_backprop` checks the resulting gradients.
- **Diamond graph trap.** For $c = a + a$ (or $c = a \cdot a$) the node a is reachable by two paths. Without `visited`, a is added twice and its backward runs twice (double counting); in a graph with deeper sharing the number of visits grows exponentially. With the set, a appears once, and the two contributions are summed by `+=`, giving $\partial c/\partial a = 2$ for $a + a$ and $2a$ for $a \cdot a$ (`test_gradient_accumulation_diamond_graph` checks that $d = (a + a) \cdot a$ gives $\partial d/\partial a = 4a = 12$).

The graph is a DAG by construction (a new node is built only from existing ones), so the DFS terminates. Because nodes are hashed by identity and no `__eq__` is overridden, two different nodes holding the same number are never confused.

4 Neural Network API

- **Neuron**: weights $w_i \sim U(-1, 1)$ and bias $b \sim U(-1, 1)$; output $\tanh(\sum_i w_i x_i + b)$. The sum is started from b so the graph has no spurious constant node.
- **Layer**: a list of `nout` neurons sharing the same input; returns a single `Value` when `nout`=1 and a list otherwise.
- **MLP**: a chain of layers built from `[nin] + nouts`. For sizes $[n_0, n_1, \ldots, n_L]$ the parameter count is $\sum_{l=1}^{L}(n_{l-1} + 1)\, n_l$; for `MLP(3,[4,4,1])` this is $16 + 20 + 5 = 41$ (checked in `test_mlp`).
- **Parameters and gradients**: each class exposes `parameters()` and `zero_grad()` (which calls `reset_grad()` on every parameter). Gradients accumulate with `+=`, so `zero_grad()` *must* be called before every backward pass in a training loop.

5 Design Justifications

- **Closures for local derivatives.** Each operator stores a `_backward` closure capturing its operands and output. This keeps the derivative rule next to its forward rule and avoids a large dispatch table keyed by operator name.
- **Accumulation with `+=`.** Required for correctness on shared nodes (Section 3.2).
- **Few primitives, many derived ops.** Only `add`, `mul`, `pow`, `tanh` and `exp` have hand-written derivatives; `neg`, `sub`, `div` and the reflected operators (`__radd__`, `__rsub__`, `__rmul__`, `__rtruediv__`) are compositions, minimizing the surface for bugs and letting expressions like `3 - a * 2 + 1 / a` work.
- **Constants are wrapped in `Value`.** `a + 1` works without special cases; the wrapped constant simply receives an unused gradient.
- **Iterative post-order DFS.** The hint's recursive DFS is correct but is limited by Python's recursion limit (about 1000 frames by default), which a long chain of operations would exceed. The engine therefore uses an explicit stack with the same post-order and `visited` logic; a chain of 20,000 additions was back-propagated without error (see the notebook).

- **Verification strategy.** Three layers: analytic hand-derived values, graph-structure tests (the diamond graph), and numerical gradient checking by central differences $\frac{f(x+h)-f(x-h)}{2h}$ with $h = 10^{-6}$ (error $\sim 10^{-10}$ in the notebook).

6 Test Results

The course test suites (`tests/test_engine.py` and `tests/test_nn.py`) contain 12 tests; all pass (`python -m pytest -v`: `12 passed`).

Test	What it checks
`test_value_initialization`	`data`, `grad`, `label`, `repr`
`test_basic_operations`	$+, -, \times, /$, negation and reflected operators
`test_type_errors`	`TypeError` for strings/lists
`test_power`	value, gradient = 12, `AssertionError` for a `Value` exponent
`test_activations_and_exp`	tanh and exp values and gradients
`test_gradient_accumulation_diamond_graph`	shared node: $d = (a + a)a \Rightarrow \partial d/\partial a = 12$
`test_chain_rule_and_backprop`	composite $f = ab(a + b)$
`test_edge_case_division_by_zero`	`ZeroDivisionError`
`test_type_casting`	`int()` and `float()`
`test_neuron`, `test_layer`, `test_mlp`	shapes, parameter counts (4, 16, 41), `zero_grad`, backprop through the network

Additional checks. Outside the course suite I also verified: (i) the analytic gradient of a composite function against central finite differences (error $\approx 10^{-10}$, in the notebook); (ii) a 20,000-node chain; and (iii) that `test_mlp`, which asserts that every parameter receives a non-zero gradient after random initialization, was not flaky: all 41 parameters had non-zero gradients for each of 3000 different random seeds.

7 Experiment: Two Moons

We train `MLP(2,[8,8,1])` (105 parameters) on 100 noisy two-moons points with labels ± 1. The loss is

$$\mathcal{L} = \frac{1}{N}\sum_i (s_i - y_i)^2 + 10^{-4}\sum_j p_j^2,$$

optimized with full-batch SGD for 200 epochs, with the learning rate decayed linearly from 0.1 to 0.01.

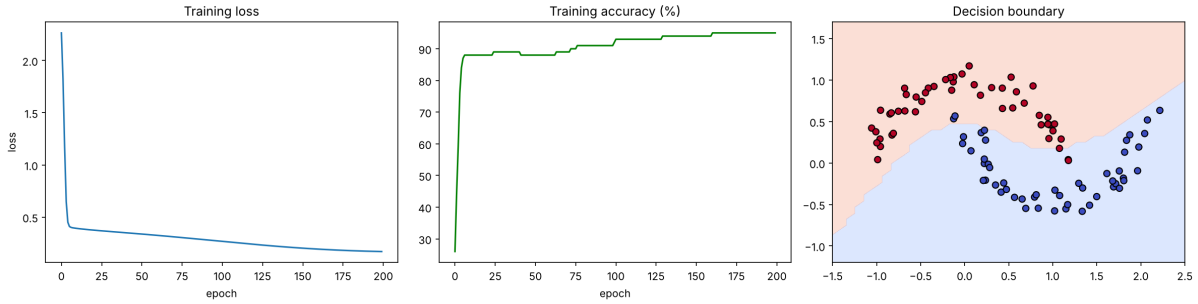


Figure 1: Training loss, training accuracy and learned decision boundary.

The loss falls from 2.26 to 0.17 and training accuracy rises from 26% to 95% (final loss 0.1703).

Observation: tanh saturation. In a first attempt with `MLP(2,[16,16,1])` and a learning rate of 0.5 the run got stuck at loss ≈ 2.01 and 50% accuracy. A loss of 2 is exactly what a constant output of $+1$ gives on balanced ± 1 labels, so the output neuron had saturated. Since $\tanh'(a) = 1 - \tanh^2(a) \to 0$ for large $|a|$, gradients vanish and the network cannot recover. Reducing the learning rate and network width fixed this (a 16×16 network with learning rate 0.05 stalled in the same way in a tuning run, confirming that it was the wide layers with $U(-1, 1)$ initialization, whose pre-activations grow with fan-in, that saturated). This illustrates why initialization scale and learning rate matter for saturating activations.

8 Conceptual Questions

Q1. Why is a topological sort needed? The gradient of a node is the sum over all of its consumers. It is final only after all consumers have run their backward step. A reverse topological order guarantees this ordering; an arbitrary order (e.g. creation order of a traversal that visits nodes twice) can use incomplete gradients.

Q2. Why `+=` instead of `=` in `_backward`? A variable used more than once (e.g. $c = a + a$, or a weight shared across samples) receives one contribution per use. Assignment would keep only the last one. With `+=`, $\partial(a + a)/\partial a = 2$ is obtained correctly.

Q3. Why must `zero_grad()` be called each step? For the same reason: gradients accumulate across calls to `backward()`. Without resetting, each update would use the sum of all previous gradients.

Q4. Forward versus reverse mode. Forward mode computes one directional derivative per pass (cost $\propto$ number of inputs for a full gradient); reverse mode computes the gradient of one scalar output with respect to *all* inputs in one backward pass (cost $\approx$ a small constant times the forward pass). Networks have millions of parameters and one scalar loss, so reverse mode (backpropagation) is the efficient choice.

Q5. Why does the seed gradient equal 1? Because $\partial L/\partial L = 1$; it is the starting value of the chain rule from which all other gradients are derived by multiplication with local derivatives.

Q6. Complexity. Both the forward and backward passes are $O(V + E)$ in the number of nodes and edges of the graph, because each node is visited exactly once thanks to the `visited` set; memory is $O(V)$ since every intermediate value and its closure is retained for the backward pass.

9 Reproducibility

```
pip install pytest numpy matplotlib nbformat nbconvert ipykernel
python -m pytest -v              # 12 passed
jupyter nbconvert --to notebook --execute demo.ipynb
```

The notebook seeds Python's `random` and NumPy with 42; the demo takes roughly 20–30 seconds.